

Laboratorio Linux – Parte 2

GREP – SED – AWK – SORT

Redes de Datos

Curso 2023

LSI – Dpto Informática FaCENA

GREP

grep significa ***G**lobally Search For **R**egular **E**xpression and **P**rint out*

Búsqueda global de expresiones regulares.

Herramienta de línea de comando usada en sistemas Linux/Unix para buscar un patrón específico en un archivo o grupo de archivos.

Posee opciones que permiten realizar varias acciones relacionadas con la búsqueda en archivos.

grep '<texto-buscado>' <archivo/archivos>

```
$ grep "DIAZ" estudiantes.csv
43335;DIAZ SOTO, ANTONIO MARIA;antony141491@hotmail.com
49581;DIAZ, Edith Nancy Adela;adediaz18@gmail.com
$
```

GREP

Tener en cuenta de usar comillas simples o dobles alrededor del texto si es más de una palabra.

Es posible usar el comodín (*) para seleccionar todos los archivos en un directorio.

El resultado son las ocurrencias del patrón (por la línea en la que se encuentra) en los archivos. Si no existe una coincidencia, no se imprimirá ninguna salida en la terminal.

```
$ grep "Leopoldo" estudiantes.csv
```

```
$
```

Opciones a usar con grep:

- **n** para indicar el número de línea de ocurrencia

```
$ grep -n "DIAZ" estudiantes.csv
2:43335;DIAZ SOTO, ANTONIO MARIA;antony141491@hotmail.com
28:49581;DIAZ, Edith Nancy Adela;adediaz18@gmail.com
$
```

Opciones a usar con grep:

- **c** imprime el número de ocurrencias en cada línea

```
$ grep -c "Romero" estudiantes.csv
```

```
4
```

```
$ grep -c "Romer" estudiantes.csv
```

```
?
```

- **v** imprime las líneas que NO coinciden con el patrón buscado

```
$ grep -v "ROMERO" estudiantes.csv
```

```
Legajo;Alumno;Contactos
```

```
50497;MERLO, VERONICA SOLEDAD;solange.veroniq@gmail.com
```

```
46789;Zapata, Sergio Ariel;sergio_compus@hotmail.com
```

```
...
```

Opciones a usar con grep:

- **i** ignora mayúsculas y minúsculas

```
$ grep -i "diaz" estudiantes.csv
```

```
27:26,49581,"DIAZ, Edith Nancy Adela",adediaz18@gmail.com,39316120
```

- **I** informa los archivos de la carpeta local que contienen el patrón buscado

```
$ grep -I "diaz" *.csv
```

```
estudiantes.csv
```

```
$
```

- **w** informa la coincidencia exacta con el patrón buscado

```
$ grep -w "ALEGRE" estudiantes.csv
```

```
5,56001,"ALEGRE, Sebastian Andres",seba_aleggre@hotmail.com,43110416
```

Opciones a usar con grep:

- **w** informa la coincidencia exacta con el patrón buscado

```
$ grep -w "DIAZ" estudiantes.csv
```

```
26,49581,"DIAZ, Edith Nancy Adela",adediaz18@gmail.com,39316120
```

```
$ grep -w "DIAZ " estudiantes.csv
```

```
$ grep -w "diaz" estudiantes.csv
```

```
$ grep -wi "diaz" estudiantes.csv
```

```
26,49581,"DIAZ, Edith Nancy Adela",adediaz18@gmail.com,39316120
```

Opciones a usar con grep:

- **-o** informa los patrones que coinciden, línea por línea

```
$ grep -o "BENITEZ" estudiantes.csv  
BENITEZ
```

```
$ grep -o -n "DIAZ" estudiantes.csv  
15:BENITEZ
```

Opciones a usar con grep:

- **A** informa la(s) coincidencias (after) despues del patrón buscado

```
$ grep -A 1 "DIAZ " estudiantes.csv
```

```
14,55527,"BENITEZ RINAS, Juan Gabriel",juanxoo321@gmail.com,43266266
```

```
15,56367,"BLANCO, FABRICIO MIGUEL",fabriblanco.03@hotmail.com,44542596
```

```
$ grep -A 1 "DIAZ" estudiantes.csv
```

```
43335;DIAZ SOTO, ANTONIO MARIA;antony141491@hotmail.com
```

```
50497;MERLO, VERONICA SOLEDAD;solange.veroniq@gmail.com
```

```
--
```

```
49581;DIAZ, Edith Nancy Adela;adediaz18@gmail.com
```

```
46834;VIRASORO, DESIREE GRISELDA;shodesi_16@hotmail.com
```


es una útil función de procesamiento de texto de GNU/Linux.

La forma completa de 'sed' es Stream Editor. Muchas tareas de procesamiento de texto simples y complejas se pueden hacer muy fácilmente usando 'sed'.

Cualquier cadena particular en un texto o un archivo se puede buscar, reemplazar y eliminar mediante el uso de una expresión regular con el comando 'sed'. En el siguiente ejemplo, 's' indica la tarea de búsqueda y reemplazo. La palabra 'Bash' se buscará en el texto, "Bash Scripting Language" y, si la palabra existe en el texto, se reemplazará por la palabra 'Perl'. í

```
$ echo "Bash Scripting Language" | sed 's/Bash/Perl/'  
"Perl Scripting Language"
```

```
$ echo "Monday Tuesday Wednesday Thursday Friday Saturday Sunday" > semana.txt
```

```
$ sed 's/Sunday/Sunday is holiday/' semana.txt  
"Monday Tuesday Wednesday Thursday Friday Saturday Sunday is holiday"
```

Opción g.

Reemplace todas las instancias de un texto en una línea particular de un archivo. Ejemplo, generamos el archivo python.txt con el siguiente texto:

```
Python is a very popular language.  
Python is easy to use. Python is easy to learn.  
Python is a cross-platform language
```

El siguiente comando reemplazará todas las apariciones de 'Python' en la segunda línea del archivo, python.txt. Aquí, 'Python' aparece dos veces en la segunda línea.

```
$ sed '2 s/Python/perl/g' python.txt
```

```
Python is a very popular language.  
perl is easy to use. perl is easy to learn.  
Python is a cross-platform language
```

```
$ sed '2 s/Python/perl/g2' python.txt
```

```
Python is a very popular language.  
Python is easy to use. perl is easy to learn.  
Python is a cross-platform language
```

Reemplazar la última coincidencia en un archivo con texto nuevo

El siguiente comando reemplazará la última aparición del patrón de búsqueda, 'Python' por el texto, 'Bash'. Aquí, el símbolo '\$' se usa para hacer coincidir la última aparición del patrón.

```
$ sed -e '$s/Python/Bash/' python.txt
```

```
Python is a very popular language.
```

```
Python is easy to use. Python is easy to learn.
```

```
Bash is a cross-platform language
```

Reemplace la ruta completa de todos los archivos con solo el nombre de archivo sin directorio.

El nombre del archivo se puede recuperar de la ruta del archivo muy fácilmente usando el comando `basename`. El comando `sed` también se puede usar para recuperar el nombre de archivo de la ruta del archivo.

```
$ pwd
```

```
/home/suse42ljr/python/mlbll-python
```

```
$ pwd | sed 's/.*/\\/'
```

```
mlbll-python
```

Agregue una cadena antes y después del patrón coincidente usando '\ 1'

```
$ echo "Bash language" | sed 's/(Bash)/Learn \1 programming/'  
Learn Bash programming language
```

Eliminar líneas de acuerdo a un patrón. Generar el archivo os.txt

```
$ sed '/OS/d' os.txt  
Windows  
Linux  
Android
```

Eliminar línea coincidente y 2 líneas después de la línea coincidente

```
$ sed '/Linux/,+2d' os.txt  
Windows
```

Eliminar todos los espacios al final de la línea de texto

```
$ sed 's/[[:blank:]]*$//' os.txt
```

Windows

Linux

Android

OS

Si hay una coincidencia en la línea, agrega algo al final de la línea

```
$ sed '/Windows/ s/$/ 10/' os.txt
```

Windows 10

Linux

Android

Si hay una coincidencia en la línea, inserta una línea antes del texto

```
$ sed '/Linux/ s/^/MS-DOS.\n/' os.txt
```

Windows

MS-DOS

Android

OS

Si hay coincidencia en la línea, inserte una línea después de esa línea

```
$ sed 's/Linux/&\nUbuntu/' os.txt
```

Windows

Linux

Ubuntu

Android

OS

Si no hay coincidencia, agrega algo al final de la línea

```
$ sed '/Linux/! s$/ Operating System/' os.txt
```

Windows Operating System

Linux

Android Operating System

OS Operating System

Si no hay una coincidencia, elimine la línea

```
$ sed '/Windows/!d' os.txt
```

Windows

Si hay coincidencia en la línea, inserte una línea después de esa línea

```
$ sed 's/Linux/&\nUbuntu/' os.txt
```

Windows

Linux

Ubuntu

Android

OS

Es una herramienta Unix/Linux de procesamiento de patrones en líneas de texto. Su utilización estándar es la de filtrar archivos o salida de comandos, tratando las líneas para mostrar una determinada información sobre las mismas.

El uso básico de AWK es:

```
awk [condicion] { comandos }
```

```
$ awk '{print $1}' semana.txt
```

```
Monday
```

```
$ awk '{print $2, $4}' semana.txt
```

```
Tuesday Thursday
```

```
$ awk '{print $0}' semana.txt
```

```
Monday Tuesday Wednesday Thursday Friday Saturday Sunday
```

```
$ awk '{ print $NF }' semana.txt
```

```
Sunday
```



```
awk [condicion] { comandos }
```

Donde:

[condicion] representa una condición de matcheo de líneas o parámetros.

comandos : serie de comandos a ejecutar:

\$0 → Mostrar la línea completa

\$1-\$N → Mostrar los campos (columnas) de la línea especificados.

FS → Field Separator (Espacio o TAB por defecto)

NF → Número de campos (fields) en la línea actual

NR → Número de líneas (records) en el stream/archivo a procesar.

OFS → Output Field Separator (" ").

ORS → Output Record Separator ("\n").

RS → Input Record Separator ("\n").

BEGIN → Define sentencias a ejecutar antes de empezar el procesado.

END → Define sentencias a ejecutar tras acabar el procesado.

length → Longitud de la línea en proceso.

FILENAME → Nombre del archivo en procesamiento.

ARGC → Número de parámetros de entrada al programa.

ARGV → Valor de los parámetros pasados al programa.

```
awk [condicion] { comandos }
```

Donde:

FS → Field Separator (Espacio o TAB por defecto)

```
$ awk '{print $0}' estudiantes.csv
```

```
Id;Legajo;Alumno;Contactos
```

```
93,46834,"Virasoro, Desiree Griselda",virasorodenise@gmail.com,379444543
```

```
94,54139,"Wanderer, Adrian German",adrianwanderer00@gmail.com,43023156
```

```
$ awk -F "," '{print $3,$4}' estudiantes.csv
```

```
"Vargas Carlos Esteban"
```

```
"Virasoro Desiree Griselda"
```

```
$ grep "Romera" estudiantes.csv | awk -F "," 'BEGIN {print "Estudiantes 2023"} {print $3,$4}'
```

```
Estudiantes 2023
```

```
"Romera Lourdes"
```

awk [condicion] { comandos }

Donde:

FS BEGIN END

```
$ grep "DIAZ" estudiantes.csv | awk -F "," 'BEGIN {print "Estudiantes 2023"} {print $3}  
END {print "Apellido DIAZ"}'
```

```
$ grep "gmail.com" estudiantes.csv | awk -F "," 'BEGIN {print "Estudiantes 2023"} {print $2}  
END {print "Con cuentas GMAIL"}'  
Estudiantes 2023
```

```
...  
Con cuentas GMAIL
```

```
awk [condicion] { comandos }
```

Funciones utilizables en las condiciones y comandos son, entre otras:

close(archivo_a_reiniciar_desde_cero)

cos(x), sin(x)

index()

int(num)

length(string)

substr(str,pos,len)

tolower()

toupper()

system(orden_del_sistema_a_ejecutar)

printf()

Los operadores soportados por awk son los siguientes:

*, /, %, +, -, =, ++, --, +=, -=, *=, /=, %=.

awk [condicion] { comandos }

Funciones utilizables toupper(), BEGIN, END

```
$ cat estudiantes.csv | awk -F "," 'BEGIN {print "Estudiantes 2023"} {print toupper($3)}  
END {print "Todos en mayusculas"}'  
Estudiantes 2023
```

```
...  
Todos en mayusculas
```

```
$ awk -F "," ' {print length($3) " " $3} ' estudiantes.csv
```

```
awk [condicion] { comandos }
```

Mostrar las líneas que contienen valores numéricos mayor o menor que uno dado:

```
$ awk -F ";" '$1 < 50000 { print $1,$2}' estudiantes.csv
```

```
$ awk -F ";" '$1 > 49999 { print $1,$2}' estudiantes.csv
```

Mostrar el estudiante con el número de Libreta Universitaria mas alto:

```
$ awk '$2 > max { max=$1; linea=$0 }; END { print linea }' estudiantes.csv
```

Utilizado para ordenar líneas de archivos de texto.

Permite ordenar alfabéticamente, en orden inverso, por número, por mes y también puede eliminar duplicados.

También puede ordenar por elementos que no estén al principio de la línea, ignorar la sensibilidad a las mayúsculas y minúsculas y devolver si un archivo está ordenado o no.

```
$ sort os.txt
```

```
Android  
Linux  
OS  
Windows
```

```
$ sort -r os.txt
```

```
Windows  
OS  
Linux  
Android
```

Orden por número “-n”

```
$ sort -n estudiantes.csv
```

Para ordenar por una columna específica y separador de campos

```
$ sort -k 2 -t ";" -f estudiantes.csv
```

Obtener los 3 registros con mayor consumo del archivo combustibles.csv

```
$ sort -k 8 -t , -n -r combustibles.csv | head -3 | awk -F "," '{ print $2, $5, $7, $8, $11}'
```